

PROYECTO 1 — DISEÑO E IMPLEMENTACIÓN

Board Game Café

1. Decisiones generales de diseño

- Usuario, Empleado y Platillo se modelan como clases abstractas, ya que van a servir de plantilla de la estructura y comportamiento para clases especiales.
- JuegoDeMesa representa el juego base o modelo conceptual del juego; JuegoVenta y JuegoPrestamo representan su presencia concreta dentro del inventario de venta y del inventario de préstamo, respectivamente.
- Se agrega Factura para representar correctamente ventas con múltiples ítems.
- Se agrega SistemaCafe como clase coordinadora de los casos de uso y de la lógica transversal.
- Se agrega PersistenciaSistema para centralizar la lectura y escritura de archivos “.txt”.
- Se usan ENUMS para estados, categorías y tipos cerrados del dominio.
- Se respetan los roles del café: las funciones exclusivas del Administrador se ubican en su clase; la coordinación general permanece en SistemaCafe.

2. Clases del sistema

Abstract Usuario

Atributos

- - login : String
- - password : String
- - nombre: String
- - juegosFavoritos : ArrayList<JuegoDeMesa>

Métodos

- + getLogin() : String
- + setLogin(login : String) : void
- + getPassword() : String
- + setPassword(password : String) : void
- + getNombre() : String
- + setNombre(nombre : String) : void
- + getJuegosFavoritos() : ArrayList<JuegoDeMesa>
- + setJuegosFavoritos(juegosFavoritos : ArrayList<JuegoDeMesa>) : void
- + agregarFavorito(juego : JuegoDeMesa) : void
- + eliminarFavorito(juego : JuegoDeMesa) : void

Justificación

Existe porque el sistema maneja varios tipos de actores que comparten autenticación e identificación básica. En el proyecto se exige que todos los usuarios tengan login y password, por lo que conviene concentrar esos datos en una superclase común.

Cliente extends Usuario

Atributos

- - idCliente : String
- - puntosFidelidad : int

Métodos

- + getIdCliente() : String
- + setIdCliente(idCliente : String) : void
- + getPuntosFidelidad() : int
- + setPuntosFidelidad(puntosFidelidad : int) : void
- + sumarPuntos(puntos : int) : void
- + redimirPuntos(puntos : int) : void

Justificación

Existe porque el cliente realiza reservas, solicita préstamos, compra productos y participa en el programa de fidelidad, por lo que necesita identidad propia y manejo de puntos.

Abstract Empleado extends Usuario

Atributos

- - idEmpleado : String

Métodos

- + getIdEmpleado() : String
- + setIdEmpleado(idEmpleado : String) : void
- + consultarTurnos() : ArrayList<Turno>
- + solicitarCambioTurno(turnoActual : Turno, tipoSolicitud : TipoSolicitud) : SolicitudCambioTurno
- + crearSugerenciaPlatillo(nombrePlatillo : String, descripcion : String) : SugerenciaPlatillo

Justificación

Existe porque los trabajadores del café comparten información laboral y participación en turnos, pero luego se especializan en funciones distintas. Además, el proyecto permite que los empleados tengan la capacidad de solicitar cambios de turno y sugerir platillos.

Mesero extends Empleado

Atributos

- - juegosQueConoce : ArrayList<JuegoDeMesa>

Métodos

- + getJuegosQueConoce() : ArrayList<JuegoDeMesa>
- + setJuegosQueConoce(juegosQueConoce : ArrayList<JuegoDeMesa>) : void
- + puedeExplicarJuego(juego : JuegoDeMesa) : boolean

Justificación

Existe porque el mesero atiende clientes, participa en caja y además puede explicar juegos difíciles cuando está capacitado.

Cocinero extends Empleado

Atributos

- sin atributos adicionales

Métodos

- sin métodos adicionales

Justificación

Existe porque el proyecto distingue este rol del mesero y exige que el café opere con al menos un cocinero por turno.

Administrador extends Usuario

Atributos

- sin atributos adicionales

Métodos

- + aprobarSolicitudCambio(solicitud : SolicitudCambioTurno) : void
- + rechazarSolicitudCambio(solicitud : SolicitudCambioTurno) : void
- + aprobarSugerencia(sugerencia : SugerenciaPlatillo) : void
- + rechazarSugerencia(sugerencia : SugerenciaPlatillo) : void
- + reabastecerInventario(idJuego : String, cantidad : int, tipoInventario : String) : void
- + moverJuegoAVentaAPrestamo(idJuego : String, cantidad : int) : boolean
- + repararJuego(idJuego : String) : void
- + reportarJuegoDesaparecido(idJuego : String) : void
- + actualizarEstadoJuego(juego : JuegoDeMesa, estado : EstadoJuego) : void
- + consultarVentasPorFecha(rangoFechas : String) : ArrayList<Venta>
- + consultarVentasPorRubro(rubro : RubroVenta) : ArrayList<Venta>
- + consultarHistorialPrestamos() : ArrayList<Prestamo>

Justificación

Existe porque el administrador controla inventarios, turnos, reportes, historial de préstamos y decisiones administrativas del negocio. Por eso sus funciones exclusivas deben quedar explícitas en su clase.

JuegoDeMesa

Atributos

- - idJuego : String
- - nombre : String
- - anioPublicacion : int
- - empresaMatriz : String
- - numJugadoresMin : int
- - numJugadoresMax : int
- - edadMinima : int
- - categoria : CategoriaJuego
- - esDifícil : boolean
- - estadoJuego : EstadoJuego

Métodos

- + getIdJuego() : String
- + setIdJuego(idJuego : String) : void
- + getNombre() : String
- + setNombre(nombre : String) : void
- + getAnioPublicacion() : int
- + setAnioPublicacion(anioPublicacion : int) : void
- + getEmpresaMatriz() : String
- + setEmpresaMatriz(empresaMatriz : String) : void
- + getNumJugadoresMin() : int
- + setNumJugadoresMin(numJugadoresMin : int) : void
- + getNumJugadoresMax() : int
- + setNumJugadoresMax(numJugadoresMax : int) : void
- + getEdadMinima() : int
- + setEdadMinima(edadMinima : int) : void
- + getCategoria() : CategoriaJuego
- + setCategoria(categoria : CategoriaJuego) : void
- + getEsDifícil() : boolean
- + setEsDifícil(esDifícil : boolean) : void
- + getEstadoJuego() : EstadoJuego
- + setEstadoJuego(estadoJuego : EstadoJuego) : void
- + esAptoParaMesa(mesa : Mesa) : boolean

Justificación

Existe porque los juegos son el centro del negocio y deben validarse según categoría, rango de jugadores, edad mínima, dificultad y estado físico. En este diseño, JuegoDeMesa representa el juego base, mientras que JuegoVenta y JuegoPrestamo representan su inventario concreto.

JuegoVenta

Atributos

- - precio : int
- - stockVenta : int

Métodos

- + getPrecio() : int
- + setPrecio(precio : int) : void
- + getStockVenta() : int
- + setStockVenta(stockVenta : int) : void
- + hayStock(cantidad : int) : boolean
- + reducirStock(cantidad : int) : void
- + aumentarStock(cantidad : int) : void

Justificación

Existe porque el contexto del proyecto separa explícitamente el inventario de juegos a la venta del inventario de préstamo. Modela la presencia concreta del juego dentro del inventario de venta.

JuegoPrestamo

Atributos

- - copiasDisponibles : int
- - copiasPrestadas : int

Métodos

- + getCopiasDisponibles() : int
- + setCopiasDisponibles(copiasDisponibles : int) : void
- + getCopiasPrestadas() : int
- + setCopiasPrestadas(copiasPrestadas : int) : void
- + hayDisponibilidad() : boolean
- + prestar() : void
- + devolver() : void
- + aumentarCopiasDisponibles(cantidad : int) : void
- + reducirCopiasDisponibles(cantidad : int) : void

Justificación

Existe porque el sistema debe controlar la disponibilidad real de los juegos prestables, sus préstamos y devoluciones. Modela la presencia concreta del juego dentro del inventario de préstamo.

Cafe

Atributos

- - nombre : String

- - direccion : String
- - capacidadMaxima : int

Métodos

- + getNombre() : String
- + setNombre(nombre : String) : void
- + getDireccion() : String
- + setDireccion(direccion : String) : void
- + getCapacidadMaxima() : int
- + setCapacidadMaxima(capacidadMaxima : int) : void
- + tieneCapacidad(cantidadPersonas : int) : boolean

Justificación

Existe porque representa el establecimiento físico y las restricciones globales, especialmente la capacidad máxima.

Mesa

Atributos

- - idMesa : String
- - disponible : boolean
- - cantidadPersonas : int
- - hayNinos : boolean
- - hayJovenes : boolean

Métodos

- + getIdMesa() : String
- + setIdMesa(idMesa : String) : void
- + getDisponible() : boolean
- + setDisponible(disponible : boolean) : void
- + getCantidadPersonas() : int
- + setCantidadPersonas(cantidadPersonas : int) : void
- + getHayNinos() : boolean
- + setHayNinos(hayNinos : boolean) : void
- + getHayJovenes() : boolean
- + setHayJovenes(hayJovenes : boolean) : void
- + esCompatibleConJuego(juego : JuegoDeMesa) : boolean
- + esCompatibleConBebida(bebida : Bebida) : boolean
- + ocuparMesa(cantidadPersonas : int, hayNinos : boolean, hayJovenes : boolean) : void
- + liberarMesa() : void

Justificación

Existe porque muchas restricciones del sistema dependen de la mesa: cantidad de personas, presencia de menores y compatibilidad con juegos o bebidas.

Reserva

Atributos

- - idReserva : String
- - estadoReserva : EstadoReserva
- - fecha : String

Métodos

- + getIdReserva() : String
- + setIdReserva(idReserva : String) : void
- + getEstadoReserva() : EstadoReserva
- + setEstadoReserva(estadoReserva : EstadoReserva) : void
- + getFecha() : String
- + setFecha(fecha : String) : void
- + confirmar() : void
- + rechazar() : void

Justificación

Existe porque el sistema debe asignar mesas y rechazar reservas cuando se excede la capacidad del café.

Prestamo

Atributos

- - idPrestamo : String
- - estadoPrestamo : EstadoPrestamo
- - fechaInicio : String
- - fechaDevolucion : String
- - advertenciaReglas : boolean

Métodos

- + getIdPrestamo() : String
- + setIdPrestamo(idPrestamo : String) : void
- + getEstadoPrestamo() : EstadoPrestamo
- + setEstadoPrestamo(estadoPrestamo : EstadoPrestamo) : void
- + getFechaInicio() : String
- + setFechaInicio(fechaInicio : String) : void
- + getFechaDevolucion() : String
- + setFechaDevolucion(fechaDevolucion : String) : void
- + getAdvertenciaReglas() : boolean

- + setAdvertenciaReglas(advertenciaReglas : boolean) : void
- + agregarJuegoAPrestamo(juego : JuegoPrestamo) : void
- + finalizarPrestamo() : void
- + tieneCupoParaMasJuegos() : boolean

Justificación

Existe porque el préstamo es una funcionalidad central del sistema y debe registrar juegos, fechas, estado y advertencias. Se renombra agregarJuego a agregarJuegoAPrestamo para hacer explícita su función.

Abstract Platillo

Atributos

- - idPlatillo : String
- - nombre : String
- - precio : int

Métodos

- + getIdPlatillo() : String
- + setIdPlatillo(idPlatillo : String) : void
- + getNombre() : String
- + setNombre(nombre : String) : void
- + getPrecio() : int
- + setPrecio(precio : int) : void

Justificación

Existe porque el café vende distintos tipos de productos que comparten identidad y precio, pero difieren en restricciones y atributos específicos.

Bebida extends Platillo

Atributos

- - alcoholica : boolean
- - caliente : boolean

Métodos

- + getAlcoholica() : boolean
- + setAlcoholica(alcoholica : boolean) : void
- + getCaliente() : boolean
- + setCaliente(caliente : boolean) : void
- + esAptaParaMesa(mesa : Mesa) : boolean

Justificación

Existe porque las bebidas tienen restricciones específicas respecto a menores y a juegos de acción.

Pasteleria extends Platillo

Atributos

- - posiblesAlergenos : ArrayList<String>

Métodos

- + getPosiblesAlergenos() : ArrayList<String>
- + setPosiblesAlergenos(posiblesAlergenos : ArrayList<String>) : void
- + informarAlergenos() : ArrayList<String>

Justificación

Existe porque el proyecto exige informar al cliente sobre posibles alérgenos en productos de pastelería.

Venta

Atributos

- - idVenta : String
- - fecha : String
- - rubro : RubroVenta
- - subtotal : double
- - impuestos : double
- - propina : double
- - total : double
- - puntosGanados : int
- - puntosRedimidos : int

Métodos

- + getIdVenta() : String
- + setIdVenta(idVenta : String) : void
- + getFecha() : String
- + setFecha(fecha : String) : void
- + getRubro() : RubroVenta
- + setRubro(rubro : RubroVenta) : void
- + getSubtotal() : double
- + setSubtotal(subtotal : double) : void
- + getImpuestos() : double
- + setImpuestos(impuestos : double) : void
- + getPropina() : double
- + setPropina(propina : double) : void
- + getTotal() : double
- + setTotal(total : double) : void
- + getPuntosGanados() : int
- + setPuntosGanados(puntosGanados : int) : void

Grupo 3

Fabio Salazar - 202511238

Luis Medina - 202321015

Nicolas Toro - 202116790

- + getPuntosRedimidos() : int
- + setPuntosRedimidos(puntosRedimidos : int) : void
- + calcularSubtotal() : double
- + calcularImpuestos() : double
- + calcularTotal() : double
- + calcularPuntos() : int

Justificación

Existe porque toda venta debe registrarse con fecha, rubro, impuestos, propina cuando aplique y puntos de fidelidad.

Factura

Atributos

- - idFactura : String
- - cantidad : int
- - precioUnitario : double
- - subtotalLinea : double

Métodos

- + getIdFactura() : String
- + setIdFactura(idFactura : String) : void
- + getCantidad() : int
- + setCantidad(cantidad : int) : void
- + getPrecioUnitario() : double
- + setPrecioUnitario(precioUnitario : double) : void
- + getSubtotalLinea() : double
- + setSubtotalLinea(subtotalLinea : double) : void
- + calcularSubtotalLinea() : double

Justificación

Existe porque una venta puede incluir varios productos, así que se necesitan ítems individuales para modelarla correctamente.

Turno

Atributos

- - idTurno : String
- - diaSemana : DiaSemana
- - horaInicio : String
- - horaFin : String

Métodos

- + getIdTurno() : String

- + setIdTurno(idTurno : String) : void
- + getDiaSemana() : DiaSemana
- + setDiaSemana(diaSemana : DiaSemana) : void
- + getHoraInicio() : String
- + setHoraInicio(horaInicio : String) : void
- + getHoraFin() : String
- + setHoraFin(horaFin : String) : void
- + esValido() : boolean

Justificación

Existe porque el sistema debe organizar turnos semanales y validar la operación mínima del negocio.

SolicitudCambioTurno

Atributos

- - idSolicitud : String
- - estado : EstadoSolicitud
- - tipoSolicitud : TipoSolicitud

Métodos

- + getIdSolicitud() : String
- + setIdSolicitud(idSolicitud : String) : void
- + getEstado() : EstadoSolicitud
- + setEstado(estado : EstadoSolicitud) : void
- + getTipoSolicitud() : TipoSolicitud
- + setTipoSolicitud(tipoSolicitud : TipoSolicitud) : void
- + aprobar() : void
- + rechazar() : void

Justificación

Existe porque los empleados pueden solicitar cambios de turno que deben ser evaluados por el administrador.

SugerenciaPlatillo

Atributos

- - idSugerencia : String
- - nombrePlatillo : String
- - descripcion : String
- - estado : EstadoSugerencia

Métodos

- + getIdSugerencia() : String
- + setIdSugerencia(idSugerencia : String) : void

- + getNombrePlatillo() : String
- + setNombrePlatillo(nombrePlatillo : String) : void
- + getDescripcion() : String
- + setDescripcion(descripcion : String) : void
- + getEstado() : EstadoSugerencia
- + setEstado(estado : EstadoSugerencia) : void
- + aprobar() : void
- + rechazar() : void

Justificación

Existe porque los empleados pueden sugerir nuevos platillos, pero la decisión final corresponde al administrador.

SistemaCafe

Atributos

- - cafe : Cafe
- - usuarios : ArrayList<Usuario>
- - juegosVenta : ArrayList<JuegoVenta>
- - juegosPrestamo : ArrayList<JuegoPrestamo>
- - mesas : ArrayList<Mesa>
- - reservas : ArrayList<Reserva>
- - prestamos : ArrayList<Prestamo>
- - platillos : ArrayList<Platillo>
- - ventas : ArrayList<Venta>
- - turnos : ArrayList<Turno>
- - solicitudesCambio : ArrayList<SolicitudCambioTurno>
- - sugerenciasPlatillo : ArrayList<SugerenciaPlatillo>

Métodos

- + getCafe() : Cafe
- + setCafe(cafe : Cafe) : void
- + getUsuarios() : ArrayList<Usuario>
- + setUsuarios(usuarios : ArrayList<Usuario>) : void
- + getJuegosVenta() : ArrayList<JuegoVenta>
- + setJuegosVenta(juegosVenta : ArrayList<JuegoVenta>) : void
- + getJuegosPrestamo() : ArrayList<JuegoPrestamo>
- + setJuegosPrestamo(juegosPrestamo : ArrayList<JuegoPrestamo>) : void
- + getMesas() : ArrayList<Mesa>
- + setMesas(mesas : ArrayList<Mesa>) : void
- + getReservas() : ArrayList<Reserva>
- + setReservas(reservas : ArrayList<Reserva>) : void

Grupo 3

Fabio Salazar - 202511238

Luis Medina - 202321015

Nicolas Toro - 202116790

- + getPrestamos() : ArrayList<Prestamo>
- + setPrestamos(prestamos : ArrayList<Prestamo>) : void
- + getPlatillos() : ArrayList<Platillo>
- + setPlatillos(platillos : ArrayList<Platillo>) : void
- + getVentas() : ArrayList<Venta>
- + setVentas(ventas : ArrayList<Venta>) : void
- + getTurnos() : ArrayList<Turno>
- + setTurnos(turnos : ArrayList<Turno>) : void
- + getSolitudesCambio() : ArrayList<SolicitudCambioTurno>
- + setSolitudesCambio(solitudesCambio : ArrayList<SolicitudCambioTurno>) : void
- + getSugerenciasPlatillo() : ArrayList<SugerenciaPlatillo>
- + setSugerenciasPlatillo(sugerenciasPlatillo : ArrayList<SugerenciaPlatillo>) : void
- + crearReserva(cliente : Cliente, cantidadPersonas : int, hayNinos : boolean, hayJovenes : boolean) : Reserva
- + realizarPrestamo(solicitante : Usuario, mesa : Mesa, listaJuegos : ArrayList<JuegoPrestamo>) : Prestamo
- + devolverPrestamo(idPrestamo : String) : boolean
- + registrarVenta(comprador : Usuario, listItems : ArrayList<Factura>, propina : double) : Venta
- # validarCapacidad(cantidadPersonas : int) : boolean
- # asignarMesa(cantidadPersonas : int) : Mesa
- # verificarRestriccionesPrestamo(mesa : Mesa, juegos : ArrayList<JuegoPrestamo>) : boolean
- # verificarDisponibilidadJuego(juego : JuegoPrestamo) : boolean
- # validarLimiteJuegos(mesa : Mesa) : boolean
- # generarAdvertenciaJuegoDifícil(juego : JuegoDeMesa) : boolean
- # calcularImpuestos(rubro : RubroVenta, subtotal : double) : double
- # calcularTotal(subtotal : double, impuestos : double, propina : double) : double
- # calcularPuntos(total : double) : int
- # validarCoberturaTurno() : boolean

Justificación

Existe porque el sistema necesita una clase coordinadora que centralice reservas, préstamos, devoluciones y ventas, sin mezclar la lógica con una interfaz completa. Las funciones exclusivas por rol se dejan en las clases del dominio correspondientes.

PersistenciaSistema

Atributos

- - rutaDatos : String

Métodos

- + getRutaDatos() : String
- + setRutaDatos(rutaDatos : String) : void
- + cargarSistema() : SistemaCafe
- + guardarSistema(sistema : SistemaCafe) : void

- + cargarUsuarios() : ArrayList<Usuario>
- + guardarUsuarios(usuarios : ArrayList<Usuario>) : void
- + cargarJuegos() : void
- + guardarJuegos() : void
- + cargarMesasReservas() : void
- + guardarMesasReservas() : void
- + cargarPrestamos() : void
- + guardarPrestamos() : void
- + cargarPlatillos() : void
- + guardarPlatillos() : void
- + cargarVentas() : void
- + guardarVentas() : void
- + cargarTurnos() : void
- + guardarTurnos() : void
- + cargarSugerencias() : void
- + guardarSugerencias() : void
- + cargarConfiguracionCafe() : Cafe
- + guardarConfiguracionCafe(cafe : Cafe) : void

Justificación

Existe porque el proyecto exige persistencia en archivos fuera de la carpeta del código fuente. Esta clase concentra la lectura y escritura de los archivos .txt sin repartir esa responsabilidad entre las entidades del dominio.

3. Enums

CategoriaJuego

- Valores: CARTAS, TABLERO, ACCION

Justificación

Existe porque el proyecto define tres categorías cerradas de juegos.

EstadoJuego

- Valores: NUEVO, BUENO, FALTA_PIEZA, DESAPARECIDO, ROBADO, EN_REPARACION

Justificación

Existe porque el estado físico y administrativo del juego debe ser consultable y gestionable.

EstadoReserva

- Valores: ACTIVA, RECHAZADA, FINALIZADA

Justificación

Existe porque una reserva puede ser aceptada, rechazada o cerrarse.

EstadoPrestamo

- Valores: ACTIVO, DEVUELTO, ATRASADO, CANCELADO

Justificación

Existe porque los préstamos requieren control de ciclo de vida e historial.

RubroVenta

- Valores: CAFETERIA, JUEGOS

Justificación

Existe porque las ventas deben discriminarse por rubros.

DiaSemana

- Valores: LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO

Justificación

Existe porque los turnos se organizan semanalmente.

EstadoSolicitud

- Valores: PENDIENTE, APROBADA, RECHAZADA

Justificación

Existe porque las solicitudes de cambio requieren evaluación administrativa.

TipoSolicitud

- Valores: CAMBIO, INTERCAMBIO

Justificación

Existe porque el proyecto distingue entre cambio general e intercambio con otro empleado.

EstadoSugerencia

- Valores: PENDIENTE, APROBADA, RECHAZADA

Justificación

Existe porque las sugerencias de platillo deben ser evaluadas.

4. Relaciones, cardinalidades y justificación

Relación	Nombre	Tipo	Cardinalidad	Justificación breve
Cliente — > Usuario	es un tipo de	Herencia	—	El cliente comparte autenticación y datos básicos con los demás usuarios, pero tiene comportamiento

Grupo 3

Fabio Salazar - 202511238

Luis Medina - 202321015

Nicolas Toro - 202116790

Relación	Nombre	Tipo	Cardinalidad	Justificación breve
				propio.
Empleado — > Usuario	es un tipo de	Herencia	—	El empleado también es un usuario autenticado del sistema.
Administrador — > Usuario	es un tipo de	Herencia	—	El administrador accede al sistema con autenticación, pero tiene privilegios distintos.
Mesero — > Empleado	es un tipo de	Herencia	—	El mesero comparte rasgos de empleado, pero tiene funciones particulares.
Cocinero — > Empleado	es un tipo de	Herencia	—	El cocinero es otra especialización del empleado.
Bebida — > Platillo	es un tipo de	Herencia	—	Representa una categoría concreta del menú.
Pastelería — > Platillo	es un tipo de	Herencia	—	Representa otra categoría concreta del menú.
Usuario ↔ JuegoDeMesa	marca como favorito	Asociación	0..n ↔ 0..n	Un usuario puede tener varios juegos favoritos y un juego puede ser favorito de varios usuarios.
Cliente ↔ Reserva	realiza	Asociación	1 ↔ 0..n	Un cliente puede realizar varias reservas a lo largo del tiempo.
Reserva ↔ Mesa	asigna	Asociación	1 ↔ 1	Cada reserva corresponde a una mesa específica.
Cliente ↔ Prestamo	solicita	Asociación	1 ↔ 0..n	Los clientes pueden solicitar préstamos durante su estadía.
Empleado ↔ Prestamo	solicita	Asociación	1 ↔ 0..n	Los empleados también pueden pedir juegos prestados bajo ciertas condiciones.

Grupo 3

Fabio Salazar - 202511238

Luis Medina - 202321015

Nicolas Toro - 202116790

Relación	Nombre	Tipo	Cardinalidad	Justificación breve
Prestamo ↔ Mesa	se asocia a	Asociación	0..1 ↔ 1	Un préstamo de cliente requiere mesa, pero uno de empleado puede no tenerla.
Prestamo ↔ Juego JuegoPrestamo	incluye	Asociación	1 ↔ 1..2	Refleja la regla de máximo dos juegos por préstamo.
Mesero ↔ Juego JuegoDeMesa	conoce	Asociación	0..n ↔ 0..n	Un mesero puede conocer varios juegos y un juego difícil puede ser conocido por varios meseros.
Empleado ↔ Turno	cumple	Asociación	1 ↔ 0..n	Un empleado puede tener varios turnos semanales o históricos.
Administrador ↔ Turno	gestiona	Asociación	1 ↔ 0..n	El administrador construye y modifica los turnos.
Empleado ↔ Solicitud SolicitudCambioTurno	presenta	Asociación	1 ↔ 0..n	Las solicitudes de cambio son realizadas por empleados.
Administrador ↔ Solicitud SolicitudCambioTurno	evalúa	Asociación	1 ↔ 0..n	El administrador las aprueba o rechaza.
Empleado ↔ Sugerencia SugerenciaPlatillo	propone	Asociación	1 ↔ 0..n	Las sugerencias son creadas por empleados.
Administrador ↔ Sugerencia SugerenciaPlatillo	decide sobre	Asociación	1 ↔ 0..n	El administrador aprueba o rechaza sugerencias.
Usuario ↔ Venta	efectúa	Asociación	1 ↔ 0..n	Clientes y empleados pueden realizar compras; el administrador queda excluido por restricción del dominio.
Cafe ↔ Juego JuegoVenta	ofrece en venta	Asociación	1 ↔ 0..n	El café mantiene inventario de juegos a la venta.
Cafe ↔ Juego JuegoPrestamo	ofrece préstamo en	Asociación	1 ↔ 0..n	El café mantiene inventario de juegos prestables.

Relación	Nombre	Tipo	Cardinalidad	Justificación breve
JuegoDeMesa ↔ JuegoVenta	se comercializa como	Asociación	1 ↔ 0..n	Un juego base puede estar representado dentro del inventario de venta.
JuegoDeMesa ↔ JuegoPrestamo	se presta como	Asociación	1 ↔ 0..n	Un juego base puede estar representado dentro del inventario de préstamo.
Cafe ↔ Platillo	ofrece	Asociación	1 ↔ 0..n	El menú pertenece al café.
Cafe ↔ Mesa	contiene	Composición	1 ↔ 1..n	Las mesas forman parte estructural del establecimiento.
Cafe ↔ Venta	registra	Asociación	1 ↔ 0..n	Todas las ventas pertenecen al negocio.
Venta ↔ Factura	se compone de	Composición	1 ↔ 1..n	Una venta está formada por uno o varios ítems.
Factura ↔ JuegoVenta	corresponde a	Asociación	0..1 ↔ 1	Una Factura de venta puede referirse a un juego vendido.
Factura ↔ Platillo	corresponde a	Asociación	0..1 ↔ 1	Una Factura de venta también puede referirse a un producto del menú.

5. Restricciones importantes del dominio

- Un cliente puede tener máximo dos juegos en préstamo al mismo tiempo.
- Un préstamo de cliente requiere mesa asignada.
- Un préstamo de empleado no requiere mesa.
- Un préstamo se bloquea si el juego no cumple edad mínima o número de jugadores.
- Si no hay mesero capacitado para un juego difícil, el préstamo puede darse con advertencia.
- Bebidas alcohólicas no pueden venderse a mesas con menores.
- Bebidas calientes no son compatibles con juegos de categoría ACCION.
- El café debe operar con al menos 1 cocinero y 2 meseros.
- Cada venta genera puntos equivalentes al 1 % del valor.
- El administrador no puede pedir juegos ni ordenar platillos.

- Todos los usuarios del sistema deben tener login y password.

6. Persistencia intermedia en .txt

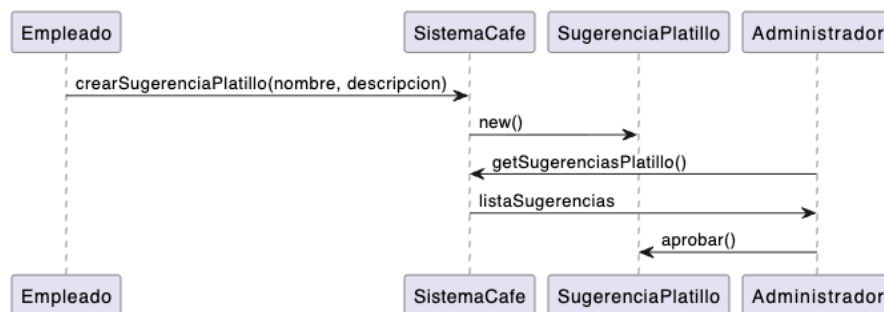
- Carpeta de persistencia: data/
- usuarios.txt
- juegos.txt
- mesas_reservas.txt
- prestamos.txt
- platillos.txt
- ventas.txt
- turnos.txt
- sugerencias.txt
- config.txt

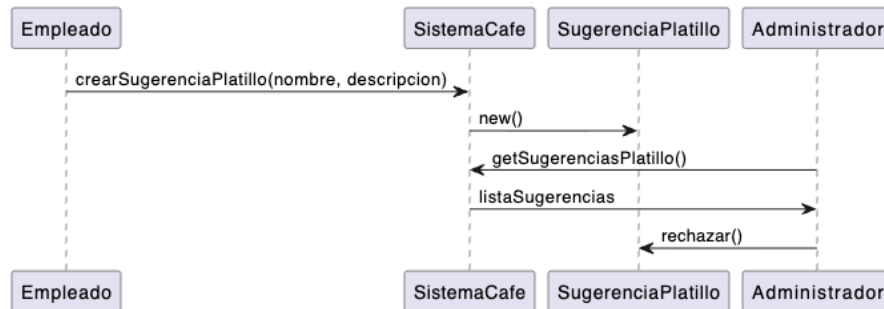
7. Métodos críticos para diagramas de secuencia

- + crearReserva(cliente, cantidadPersonas, hayNinios, hayJovenes) : Reserva
- + realizarPrestamo(solicitante, mesa, listaJuegos) : Prestamo
- + devolverPrestamo(idPrestamo) : boolean
- + registrarVenta(comprador, listItems, propina) : Venta
- + solicitarCambioTurno(empleado, turnoActual, tipoSolicitud) : SolicitudCambioTurno
- + moverJuegoAVentaAPrestamo(idJuego, cantidad) : boolean

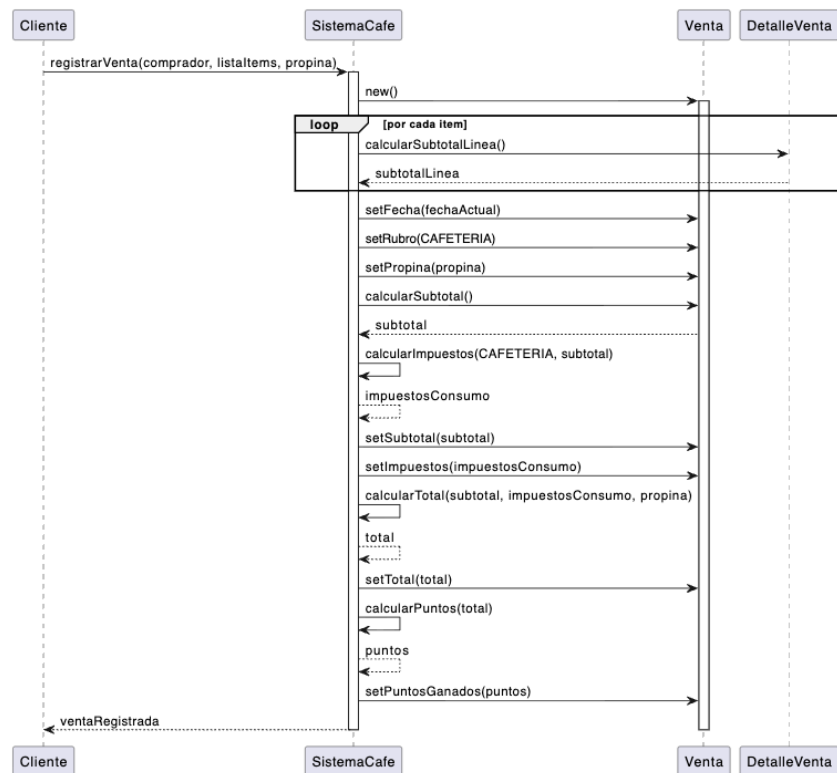
Requerimiento: Los empleados pueden sugerir nuevos platillos, pero su aprobación depende del administrador.

Caso 1: el empleado sugiere un platillo y el administrador lo aprueba.



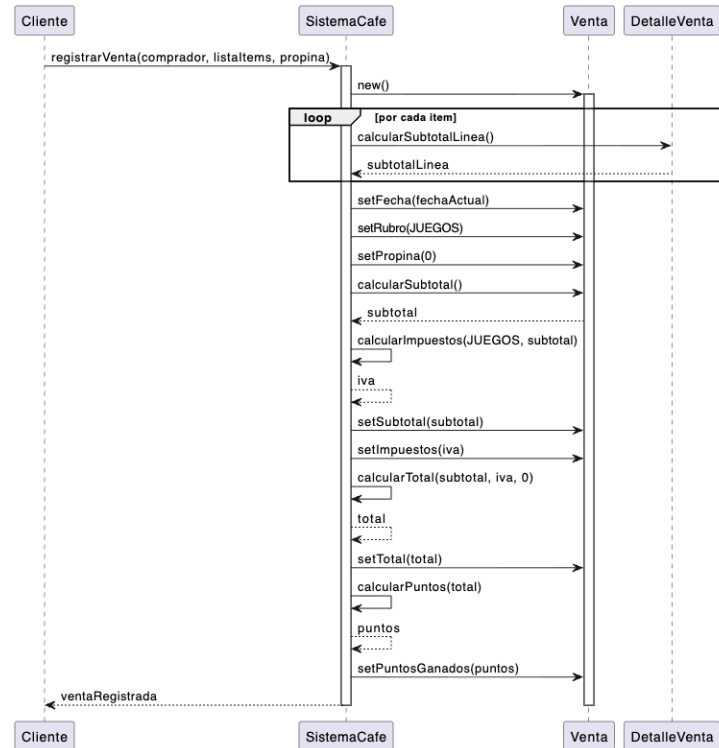
Caso 2: el empleado sugiere un platillo y el administrador no lo aprueba.

Requerimiento: Las ventas de cafetería deben contemplar impuesto al consumo y propina configurable, mientras que las ventas de juegos deben contemplar IVA. Además, toda venta debe quedar registrada con sus detalles, fecha y rubro correspondiente.

Caso 1: Ventas de Cafetería

El sistema registra la venta creando un objeto Venta y procesando cada DetalleVenta para calcular el subtotal. Luego asigna el rubro CAFETERIA, calcula el impuesto al consumo, agrega la propina configurada por el cliente y calcula el total final. Finalmente, calcula los puntos y registra la venta con fecha, rubro y detalles.

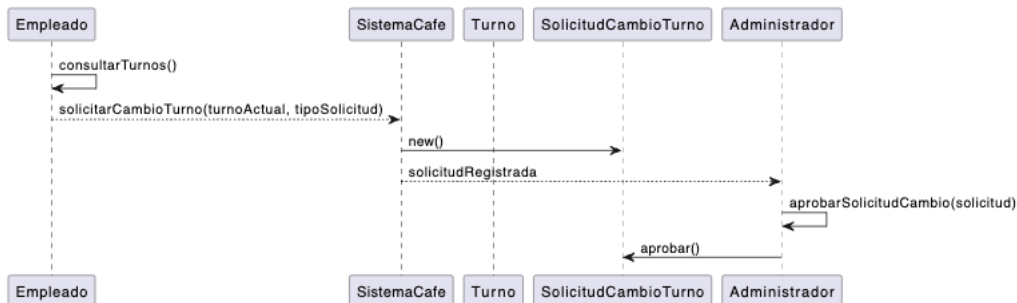
Caso 2: Venta de Juegos



El sistema registra la venta creando un objeto Venta y procesando cada DetalleVenta. Después asigna el rubro JUEGOS, calcula el IVA correspondiente y calcula el total final. Finalmente, calcula los puntos y deja registrada la venta con sus detalles, fecha y rubro.

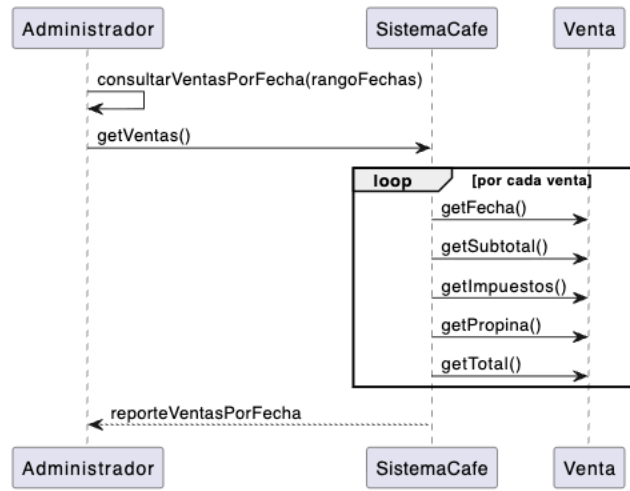
Requerimiento: El administrador también debe poder gestionar turnos, aprobar cambios y consultar reportes detallados de ventas por rubro, fecha, costos, impuestos y propinas.

Caso 1: Empleado solicita cambio de turno y el administrador lo aprueba



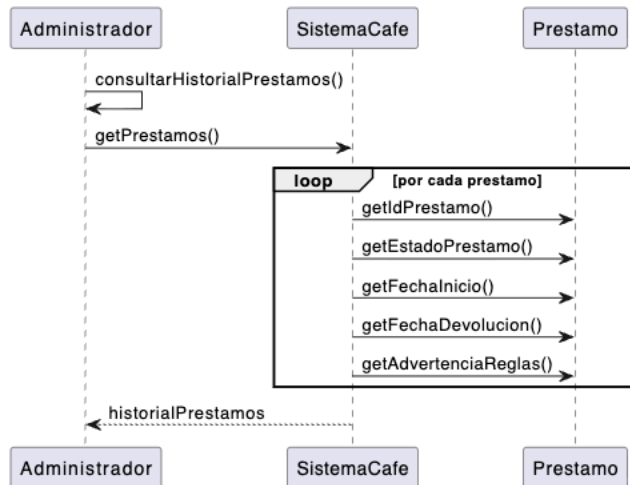
El empleado consulta sus turnos y solicita un cambio sobre uno de ellos. El sistema registra la solicitud y la pone a disposición del administrador. Si el administrador la aprueba, la solicitud cambia a estado aprobada.

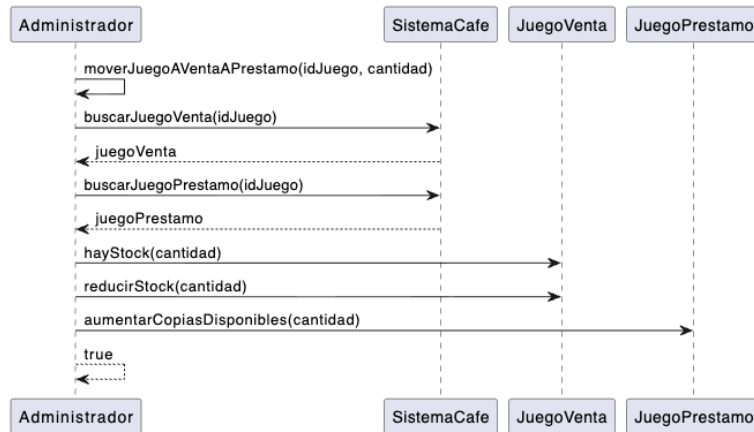
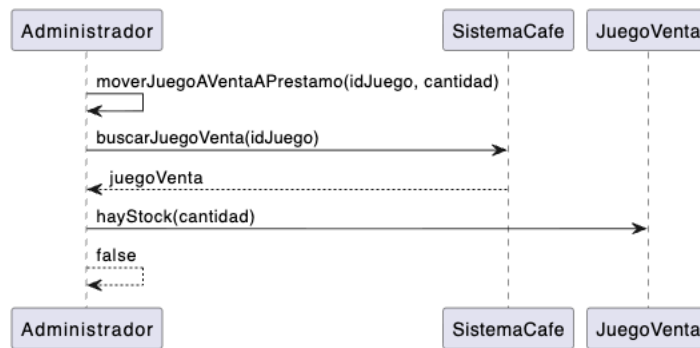
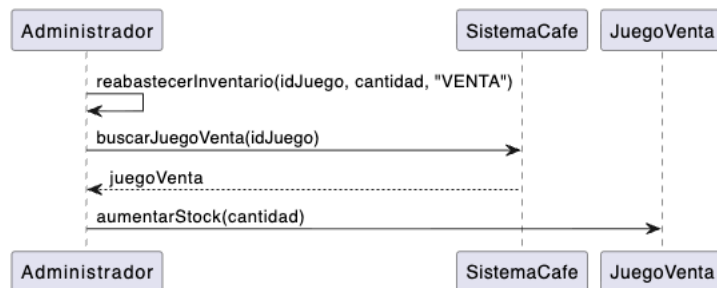
El administrador solicita un reporte por rubro. El sistema revisa las ventas registradas y selecciona aquellas cuyo rubro coincida. Después consulta los valores de subtotal, impuestos, propina y total de cada venta para construir el reporte.

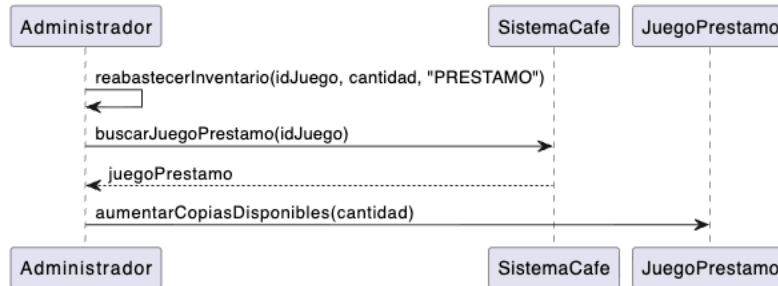
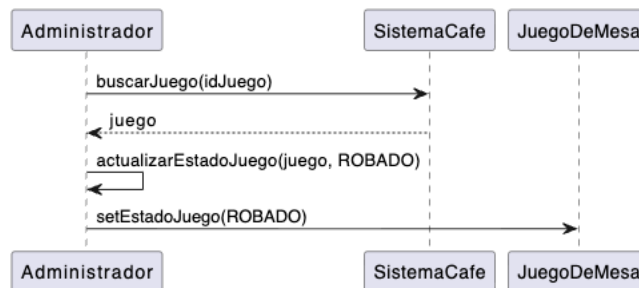
Caso 4: Administrador consulta ventas por fecha

El administrador solicita un reporte por rango de fechas. El sistema filtra las ventas de acuerdo con la fecha y consulta sus valores económicos para generar el reporte.

Requerimiento: El administrador debe poder consultar el historial de préstamos, mover juegos entre inventarios, reabastecer inventarios, reparar juegos y reportar juegos como desaparecidos o robados cuando no se devuelvan a tiempo.

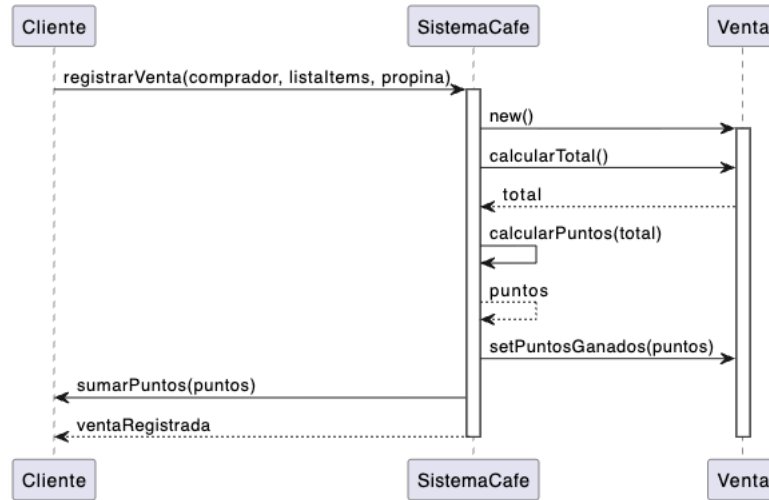
Caso 1: Consultar historial de préstamos

Caso 2: Mover juego de venta a préstamo cuando hay stock suficiente**Caso 3: Mover juego de venta a préstamo cuando no hay stock suficiente****Caso 4: Reabastecer inventario de Venta**

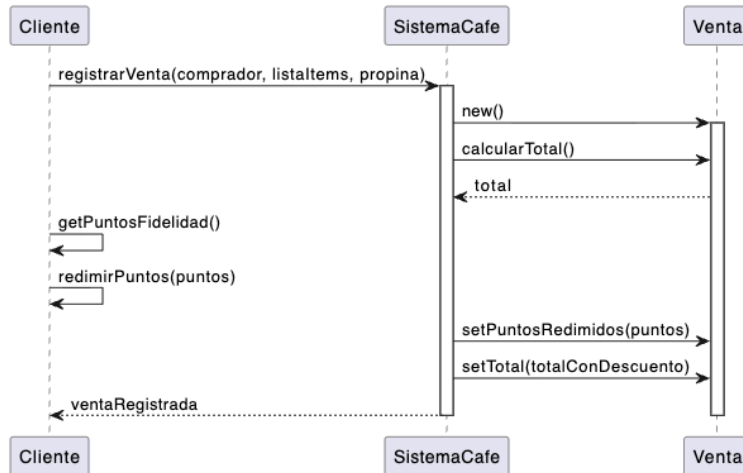
Caso 5: Reabastecer inventario de Préstamo**Caso 6: Reparar Juego****Caso 7: Reportar juego como desaparecido****Caso 8: Reportar juego como robado**

Requerimiento: Cada venta genera puntos de fidelidad equivalentes al 1% de su valor, y cada punto equivale a un peso que puede usarse como descuento en compras futuras

Caso 1: La venta genera puntos de fidelidad



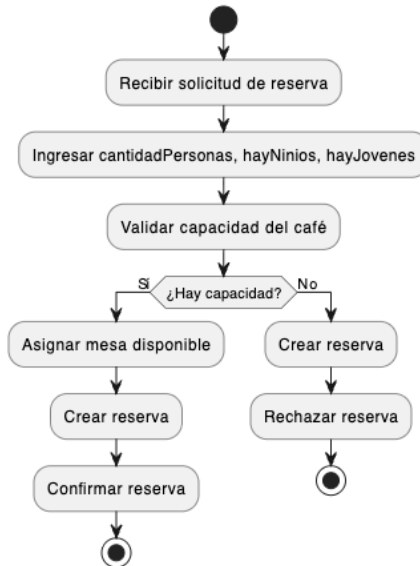
Caso 2: El cliente usa puntos como descuento en una compra futura



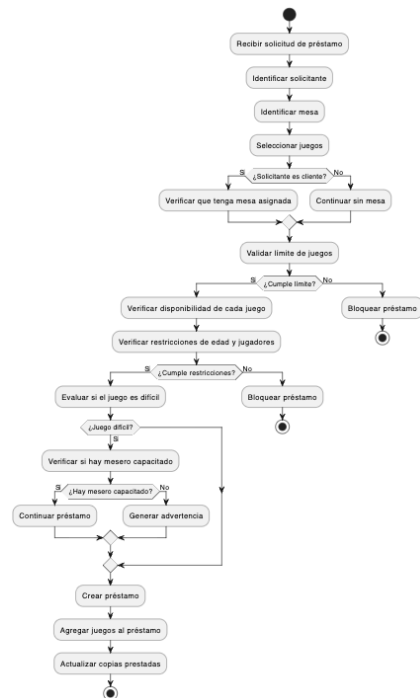
8. Diagramas de Actividades Frecuentes

Los siguientes diagramas se realizaron para mostrar el funcionamiento del sistema en sus procesos críticos y mas frecuentes.

Actividad 1: Crear Reserva



Actividad 2: Realizar Préstamo



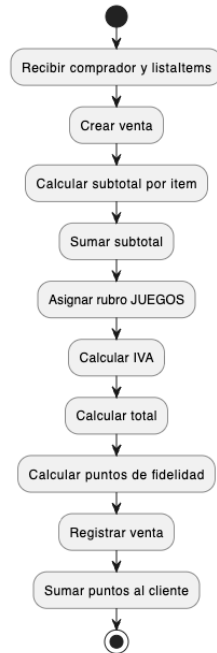
Grupo 3

Fabio Salazar - 202511238

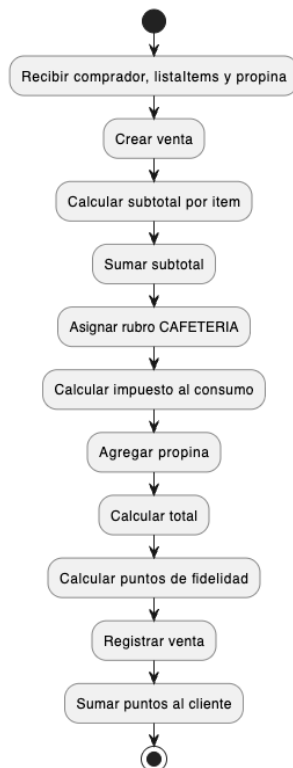
Luis Medina - 202321015

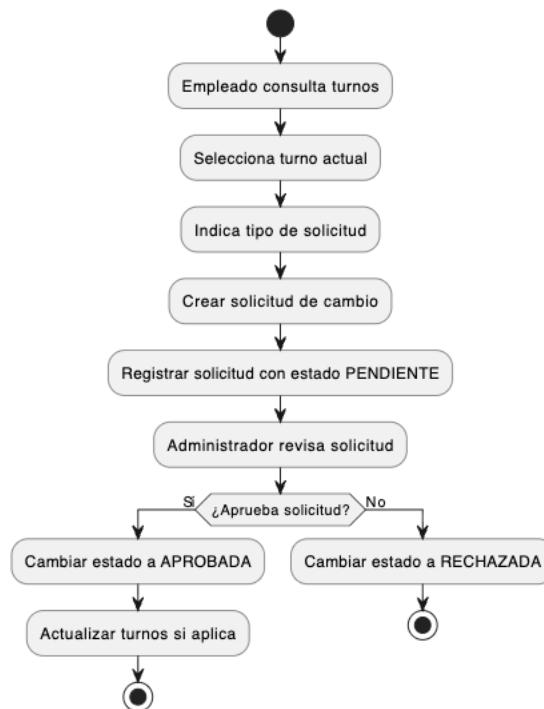
Nicolas Toro - 202116790

Actividad 3: Registrar Ventas de Juegos



Actividad 4: Registrar Ventas de Cafetería



Actividad 5: Solicitar cambio de turno

9. Justificación de Decisiones de Diseño

A continuación, se justifican las principales decisiones de diseño tomadas en el proyecto.

En primera instancia, se decidió crear la clase SistemaCafe como clase principal del sistema, en la cual se coordinan las operaciones generales, como venta, préstamos y reservas, de la cafetería. Esto se hizo para seguir el principio de controlador, donde una clase se encarga de gestionar las operaciones principales del sistema.

También se tomó la decisión de separar venta y el detalle de la venta. Es decir, se separó la información general de la venta (fecha, subtotal, impuestos y rubro) y por otro lado los detalles donde se detallan los productos, cantidades y precios. Esto se hizo con el fin de poder registrar varios productos en una sola venta, calcular subtotales por producto y poder escalar el modelo, sabiendo que en una misma venta puede haber múltiples productos.

Del mismo modo, se utilizaron enumeraciones dentro de los paquetes, debido a que estos permiten controlar los valores válidos y evitar posibles errores de Strings. Esto se hizo debido a que esta información es muy importante en clases que manejan estados, como reservas y préstamos, y se debe tener control total sobre estos estados, en especial al momento de escalar el sistema.

Finalmente, se tomó la decisión de separar los inventarios de los juegos. Se separaron los inventarios de cafetería para préstamos y los de venta. Se debe a que el stock de ambos se maneja de manera diferente y con una frecuencia distinta, por lo que al separarlos hay un mejor control sobre el flujo del producto. Además, los productos tienen impuestos diferentes y tienen movimientos dentro de sí mismos diferentes.

10. Uso de IA

Escrito por Fabio Salazar:

En algunas clases se ve un override de ToString(). Esto lo descubrí buscando una forma de pulir los datos retornados. Y, con ayuda de ChatGPT, logré implementarlos en algunas clases. No está en todas porque, honestamente, no nos alcanzó el tiempo.

Usé ChatGPT para confirmar si mi inicialización de la clase Main estaba bien hecha.

Finalmente, también fue usado para revisar algunos detalles de la propia lógica de las clases y del sistema. Sin embargo, las partes críticas del proyecto que me correspondían fueron hechas por mí.

Escrito por Luis Medina:

Utilice ChatGPT para generar el código que realizaría los diagramas de secuencia, dándole la secuencia de clases y métodos que se utilizaban. Esto con el fin de utilizar la extensión PlantUML, para obtener una mejor visualización y estandarizar la estructura de todos los diagramas.

Con respecto al main:

Intentamos de muchas maneras hasta quedarnos sin tiempo, por lo que se usó a Claude para que nos ayudara con la creación de una consola. Nuestra confusión fue que intentamos crear una interfaz en vez de hacer algo más sencillo. La clase de LectorArchivos era de la interfaz anterior, que usaba los escenarios de prueba guardados en la carpeta “datos de prueba” para comprobar que la lógica, hecha por nosotros, funcionase. Aunque esa misma clase fue hecha con ayuda de ChatGPT, dado a que no teníamos experiencia previa en leer archivos txt, solo de lecturas csv en python.